

Experiment-1A

Name: Challapalli Kodanda Rama Sai Teja

Rollno:24485A0503

Aim:Write a program to understand simple and special datatypes.

Description:

TypeScript is a strongly typed superset of JavaScript. It supports all JavaScript primitive types and also introduces additional special types to improve type safety and flexibility. These types form the foundation for building robust and scalable applications.

TypeScript Simple Types

JavaScript and TypeScript Primitives

The most basic types in TypeScript are called **primitives**.

These types form the building blocks of more complex types in your applications.

TypeScript includes all JavaScript primitives plus additional type features.

Boolean

- Represents true/false values.
- Used for flags, toggles, and conditions.

Number

- Represents both integers and floating-point numbers.
- TypeScript uses the same number type for all numeric values

String

- Represents text data.
- Can use single quotes ('), double quotes ("), or backticks (`) for template literals

Program:

```
let rollno: string = '24485A0503';
let sname: string = 'SaiTeja';
let branch: string = 'CSE';
let section: string = 'A';
let age: number = 19;
let cgpa: number = 9.22;
let phoneno: number = 9390446548;
let isstudent: boolean = true;
console.log('Student Rollno: ' + rollno);
console.log('Student Name: ' + sname);
console.log('Student Branch: ' + branch);
console.log('Student Section: ' + section);
console.log('Student Age: ' + age);
console.log('Student CGPA: ' + cgpa);
console.log('Student Phone Number: ' + phoneno);
console.log('Is Student: ' + isstudent);
```

Output:

```
C:\Users\Admin>E:

E:\>cd 24485A0503\Experiment-1

E:\24485A0503\Experiment-1>npm install typescript --save -dev
npm warn config dev Please use --include=dev instead.

added 1 package in 13s

E:\24485A0503\Experiment-1>npx tsc 1a.ts

E:\24485A0503\Experiment-1>node 1a.js
Student Rollno:24485A0503
Student Name:SaiTeja
Student Branch:CSE
Student Section:A
student Age:19
Student CGPA:9.22
Student Phone Number:9390446548
Is Student:true

E:\24485A0503\Experiment-1>|
```

Experiment-1B

Aim: Write about the typescript functions and it's types

Description:

TypeScript Functions

1. Return Type

The **return type** of a function specifies the type of value that the function sends back after execution. It helps TypeScript ensure that the function returns the correct type of data. Declaring return types improves code clarity and prevents unexpected errors.

Program:

```
function getTime():number{
    return new Date().getTime();
}
console.log(getTime());
```

Output:

```
E:\24485A0503\Experiment-1>npx tsc 1b-1.ts
E:\24485A0503\Experiment-1>node 1b-1.js
1764756069280
E:\24485A0503\Experiment-1>|
```

2. Void Return Type

The **void return type** is used for functions that do not return any value. Such functions perform actions like displaying output or modifying data but do not produce a result.

Program:

```
function printHello():void{
    console.log("Hello I'm Sai Teja");
}
printHello();
```

Output:

```
E:\24485A0503\Experiment-1>npx tsc 1b-2.ts
E:\24485A0503\Experiment-1>node 1b-2.js
Hello I'm Sai Teja
E:\24485A0503\Experiment-1>|
```

3. Parameters

Parameters are variables defined in a function that receive values when the function is called. In TypeScript, parameters can be assigned specific data types, ensuring that correct values are passed to the function.

Program:

```
function multiply(a:number,b:number){
    return a*b;
```

```
}  
console.log('Multiplication of 10 and 7 is:'+multiply(10,7));
```

Output:

```
E:\24485A0503\Experiment-1>npx tsc 1b-3.ts  
  
E:\24485A0503\Experiment-1>node 1b-3.js  
Multiplication of 10 and 7 is:70
```

4. Optional Parameters

Optional parameters are parameters that may or may not be provided when calling a function. They allow flexibility in function usage. Optional parameters must be placed after required parameters to avoid ambiguity.

Program:

```
function add(a:number,b:number,c?:number){  
    return a+b+(c|0);  
}  
console.log('Without use of third argument:'+add(3,50));  
console.log('With use of third argument:'+add(3,50,100));
```

Output:

```
E:\24485A0503\Experiment-1>npx tsc 1b-4.ts  
  
E:\24485A0503\Experiment-1>node 1b-4.js  
Without use of third argument:53  
With use of third argument:153  
  
E:\24485A0503\Experiment-1>
```

5. Default Parameters

Default parameters are parameters that have predefined values. If no value is provided during the function call, the default value is automatically used.

Program:

```
function pow(value:number,exponent:number=3){  
    return value**exponent;  
}  
console.log('Power value is:'+pow(10));
```

Output:

```
E:\24485A0503\Experiment-1>npx tsc 1b-5.ts  
  
E:\24485A0503\Experiment-1>node 1b-5.js  
Power value is:1000  
  
E:\24485A0503\Experiment-1>
```

6. Named Parameters

TypeScript does not directly support named parameters as a language feature. However, named parameters can be achieved by passing an object to a function.

Program:

```
function divide({dividend,divisor}:{dividend:number,divisor:number}){  
    return dividend/divisor;  
}  
console.log('Divison of 50 and 5:'+divide({dividend:50,divisor:5}));
```

Output:

```
E:\24485A0503\Experiment-1>npx tsc 1b-6.ts  
  
E:\24485A0503\Experiment-1>node 1b-6.js  
Divison of 50 and 5:10  
  
E:\24485A0503\Experiment-1>
```

7. Rest Parameters

Rest parameters allow a function to accept an unlimited number of arguments. All extra arguments are grouped into a single parameter. This is useful when the number of inputs is not fixed. Rest parameters must always be placed at the end of the parameter list.

Program:

```
function  
add(a:number,b:number,...rest:number[]){  
    return a+b+rest.reduce((p,c)=>p+c,0);  
}  
console.log(add(10,10,10,10,10));
```

Output:

```
npx notice  
D:\24485A0503\Experiment-1>npx tsc 1b-7.ts  
D:\24485A0503\Experiment-1>node 1b-7.js  
50  
D:\24485A0503\Experiment-1>
```

Experiment-1C

Aim: Write a program to show the importance of arrow functions use Optional, Default, REST parameters.

Description:

Arrow Functions

Arrow functions provide a shorter and more concise syntax for writing functions. They are widely used in modern TypeScript applications, especially in callbacks and functional programming.

1)Optional Parameters

Optional parameters allow functions to be called with fewer arguments than defined.

Program:

```
const greetUser=(name?:string):string=>{
    return name?'Hello,${name}!':"Hello,Guest!";
};
console.log(greetUser("SaiTeja"));
console.log(greetUser());
```

Output:

```
E:\24485A0503\Experiment-1>npx tsc 1c-1.ts

E:\24485A0503\Experiment-1>node 1c-1.js
Hello, SaiTeja!
Hello, Guest!
```

2)Default Parameters

Default parameters assign predefined values to function parameters when no argument is passed.

Program:

```
const calculateArea=(length:number,width:number=10):number=>{
    return length * width;
};
console.log(calculateArea(5));
console.log(calculateArea(5,40));
```

Output:

```
E:\24485A0503\Experiment-1>npx tsc 1c-2.ts

E:\24485A0503\Experiment-1>node 1c-2.js
50
200
```

3)REST Parameters

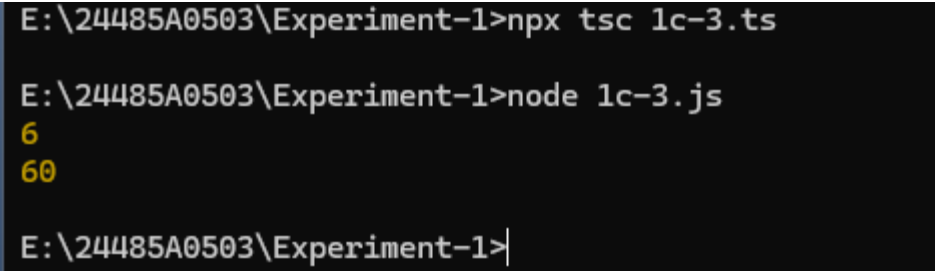
Rest parameters allow a function to accept multiple arguments as a single grouped parameter.

Program:

```
const sumAll = (...numbers: number[]): number => {  
    return numbers.reduce((total, num) => total + num, 0);  
};
```

```
console.log(sumAll(1, 2, 3));  
console.log(sumAll(10, 20, 30));
```

Output:



```
E:\24485A0503\Experiment-1>npx tsc 1c-3.ts  
  
E:\24485A0503\Experiment-1>node 1c-3.js  
6  
60  
  
E:\24485A0503\Experiment-1>|
```

Experiment-1D

Aim: Write a program to understand the working of typescript with class, constructor, properties, methods, access specifiers

Description:

Class in TypeScript

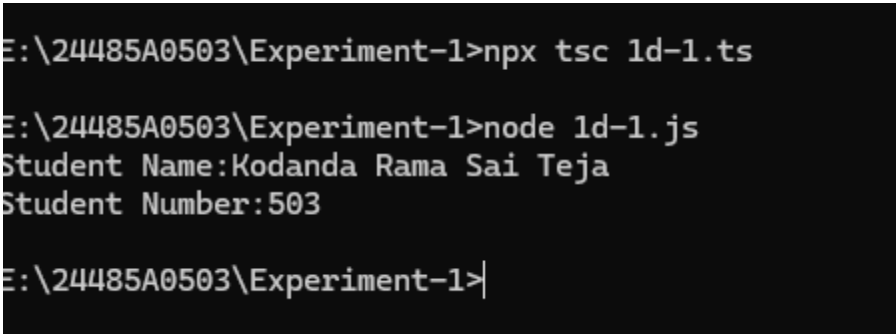
A **class** is a user-defined data type that represents a real-world entity. It groups related data (properties) and behaviors (methods) into a single unit.

Program:

```
class Student {  
    name: string;  
    sno: number;  
}
```

```
const stu = new Student();  
stu.name = "Kodanda Rama Sai Teja";  
stu.sno=503;  
console.log('Student Name:'+stu.name);  
console.log('Student Number:'+stu.sno);
```

Output:



```
E:\24485A0503\Experiment-1>npx tsc 1d-1.ts  
  
E:\24485A0503\Experiment-1>node 1d-1.js  
Student Name:Kodanda Rama Sai Teja  
Student Number:503  
  
E:\24485A0503\Experiment-1>
```

Constructor

A **constructor** is a special method that is automatically invoked when an object of a class is created. It is mainly used to initialize class properties.

Program:

```
class Student {  
    name: string;  
    sno: number;  
    constructor(name:string,sno:number)  
    {  
        this.name=name;  
        this.sno=sno;  
    }  
    public getData()  
    {
```



```
    console.log('Student Name is:'+this.name);  
    console.log('Student Number:'+this.sno);  
  }  
}
```

```
const stu = new Student('Sai Teja' , 503);  
stu.getData();
```

Output:

```
E:\24485A0503\Experiment-1>npx tsc 1d-2.ts  
  
E:\24485A0503\Experiment-1>node 1d-2.js  
Student Name is:Sai Teja  
Student Number:503  
  
E:\24485A0503\Experiment-1>
```

Properties: Properties are variables inside a class that define the characteristics of the class. They store the state of the class.

Properties can be public, private, or protected, which influences how they can be accessed outside the class.

Methods: Methods are functions defined inside a class. They define the behaviors of the class.

Methods can operate on properties and perform actions like calculations or processing.

Access Specifiers: Access specifiers control the visibility and accessibility of class members (properties and methods). TypeScript supports three access specifiers:

o **public:** Members are accessible from anywhere, both inside and outside the class.

o **private:** Members are only accessible within the class where they are defined.

o **protected:** Members are accessible within the class and its subclasses, but not outside the class.

Program:

```
class Student {  
  public name: string;  
  private age: number;  
  protected course: string;  
  constructor(name: string, age: number, course: string) {  
    this.name = name;  
    this.age = age;  
    this.course = course;  
  }  
  public displayDetails(): void {
```

```
        console.log("Student Name: " + this.name);
        console.log("Age: " + this.age);
        console.log("Course: " + this.course);
    }
    private getAgeInMonths(): number {
        return this.age * 12;
    }
    public showAgeInMonths(): void {
        console.log("Age in months: " + this.getAgeInMonths());
    }
}
class GraduateStudent extends Student {
    public showCourse(): void {
        console.log("Course (protected): " + this.course);
    }
}
let stu = new Student("Sai Teja", 19, "Computer Science");
stu.displayDetails();
stu.showAgeInMonths();
let grad = new GraduateStudent("Krish", 20, "Quantum Computing");
grad.showCourse();
```

Output:

```
E:\24485A0503\Experiment-1>npx tsc 1d-3.ts
E:\24485A0503\Experiment-1>node 1d-3.js
Student Name: Sai Teja
Age: 19
Course: Computer Science
Age in months: 228
Course (protected): Quantum Computing
E:\24485A0503\Experiment-1>
```

Experiment-1E

Aim: Write a program to understand the working of namespace and modules.

Description:

Namespace

A namespace is used to group related variables, functions, and classes under one name.

It helps avoid naming conflicts in large code bases.

Namespaces are mainly used in internal or legacy TypeScript projects.

They work within a global or shared scope.

Namespaces are less preferred in modern application development.

Program:

```
namespace Geometry {  
  export class Rectangle {  
    private width: number;  
    private height: number;  
  
    constructor(width: number, height: number) {  
      this.width = width;  
      this.height = height;  
    }  
  
    public area(): number {  
      return this.width * this.height;  
    }  
  }  
  export class Circle {  
    private radius: number;  
  
    constructor(radius: number) {  
      this.radius = radius;  
    }  
  
    public area(): number {  
      return Math.PI * this.radius * this.radius;  
    }  
  }  
}  
  
let rect = new Geometry.Rectangle(50,7);  
console.log("Rectangle Area:", rect.area());  
let circle = new Geometry.Circle(10);  
console.log("Circle Area:", circle.area());
```

Output:

```
E:\24485A0503\Experiment-1>npx tsc 1e-1.ts  
E:\24485A0503\Experiment-1>node 1e-1.js  
Rectangle Area: 350  
Circle Area: 314.1592653589793
```

Modules

A module represents a file with its own scope in TypeScript. It helps organize code across multiple files. Modules use import and export to share functionality. They support scalability and better dependency management. Modules are recommended for modern TypeScript applications.

Program:

```
export function add(a: number, b: number): number {  
    return a + b;  
}
```

```
export function multiply(a: number, b: number): number {  
    return a * b;  
}
```

```
export class Calculator {  
    subtract(a: number, b: number): number {  
        return a - b;  
    }  
}
```

```
import { add, multiply, Calculator } from "./mathUtils";
```

```
console.log("Addition:", add(10, 5));  
console.log("Multiplication:", multiply(40, 100));
```

```
let calc = new Calculator();  
console.log("Subtraction:", calc.subtract(10, 7));
```

Output:

```
E:\24485A0503\Experiment-1>npx tsc 1e-2.ts  
E:\24485A0503\Experiment-1>node 1e-2.js  
Addition: 15  
Multiplication: 4000  
Subtraction: 3
```

Experiment-1F

Aim: Write a program to understand generics, variables, functions and constraints.

Description:

Generics in TypeScript allow developers to write reusable and flexible code that works with different data types while maintaining type safety. Instead of working with a specific type, generics use type parameters to represent unknown types that are specified later.

Generic Variables

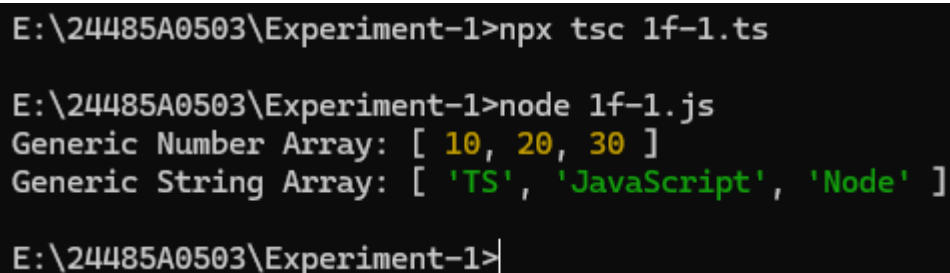
Generic variables allow a variable to hold values of different data types in a type-safe manner. The actual data type is defined when the variable is used, not when it is declared. This helps in writing reusable code without losing type checking.

Program:

```
let genericNumber: Array<number> = [10, 20, 30];  
let genericString: Array<string> = ["TS", "JavaScript", "Node"];
```

```
console.log("Generic Number Array:", genericNumber);  
console.log("Generic String Array:", genericString);
```

Output:



```
E:\24485A0503\Experiment-1>npx tsc 1f-1.ts  
  
E:\24485A0503\Experiment-1>node 1f-1.js  
Generic Number Array: [ 10, 20, 30 ]  
Generic String Array: [ 'TS', 'JavaScript', 'Node' ]  
  
E:\24485A0503\Experiment-1>
```

Generic Functions

Generic functions are functions that operate on a variety of data types instead of a single specific type. The function logic remains the same, but the data type is provided when the function is called.

Program:

```
function displayItem<T>(item: T): void {  
    console.log("Item:", item);  
}  
  
displayItem<number>(50);  
displayItem<string>("Hello Generics Template ");  
displayItem<boolean>(true);
```

Output:

```
E:\24485A0503\Experiment-1>npx tsc 1f-2.ts
```

```
E:\24485A0503\Experiment-1>node 1f-2.js
```

```
Item: 50
```

```
Item: Hello Generics Template
```

```
Item: true
```

```
E:\24485A0503\Experiment-1>
```

Generic Functions with multiple parameters

```
function pair<T, U>(first: T, second: U) {
```

```
    return { first, second };
```

```
}
```

```
console.log("Pair:", pair<string, number>("Age", 19));
```

```
E:\24485A0503\Experiment-1>npx tsc 1f-3.ts
```

```
E:\24485A0503\Experiment-1>node 1f-3.js
```

```
Pair: { first: 'Age', second: 19 }
```

```
E:\24485A0503\Experiment-1>
```